



Mo Tu We Th ☒ Sa Su

"SE"

Memo No.

Date 27th Sep / 2024

4A

lec1:

Software Engineering

concerned with all aspects of software production

= Why software Engineering?

concerned with theories, methods & tools for professional SD.

→ engineering discipline

= What is software? (FAQ)

software Developments

computer program & associated documentation such as requirements, design models & user manuals. (it maybe developed for a particular person or a general market)

→ Software may products as:

- Generic (any one) e.g p.c, word
- Custom (special dept)
↳ e.g Air traffic control traffic monitoring

Computer Sciences

concerned with theory & fundamentals.

Software Engineering.

concerned with practicality of developing & delivering useful software.

// Software Process.

Development or evolution of software

Set of activities: goals. (Specification, Development, Validation, Evolution) 4 steps in process.

- Generic activities:

Specification (system should do) Evolution. (changing)

Development (production of software) (production of SS)

Validation (checking that fulfill the customer needs)



// Software Process Models.

* Specific Perspective:

- Workflow perspective - sequence of activities
- Data flow perspective - info flow
- Role/action perspective - who does what.

* Generic Process Models:

- Water fall.
- Incremental development.
- component-based SE

Software

SE: Computer-based system development like hardware, software & process engineering. This process concerned with developing the software infrastructure controls, applications and database in the system.

SS System Engineering: involved in system specification, architectural design, integration & ~~development~~ deployment.

Software Engineering Methods:

includes system models, notations, Rules, ^{design} advice, ^{what activity to follow} process guidance



Mo Tu We Th Fr **Sa** Su

Memo No. _____

Date **28 / Sep / 2024**

lec 2:

SE

is the use of Software tools to help create & maintain ^{complex} ^{prog}

= **CASE** (Computer Aided Software Engineering)

→ Programming, debugging.

- lower case (later activities, e.g. testing, checking etc.)
- upper case. (modeling), (design, requirement)...

= Attributes of good Software:

- Maintainability → must evolve to meet new change.
- Dependability → Reliability, Security, safety
- Efficiency → responsiveness, process time, memory utilization
- Acceptability → user accept your software (easy to use).
↳ must be understandable / compatible with other soft

= Key challenges facing SE:

- Heterogeneity (any type of devices or net it works.)
- Business and Social Change (changes with time)
- Security and trust.
- X [Increasing demands.] X

= Reasons for software failure:

- Increasing demands.



Mo Tu We Th Fr Sa Su

Memo No. _____

Date ____/____/____

- Lack of SE techniques.

= SE diversity.

Depend on each other.

1. Type of Applications.

2. user requirements.

3. developer background (team)

= Application types:

- Stand-alone application (work without network)

- Interactive transaction-based application (Remotely used)

- Embedded control system (Manual thing with

→ includes web applications such as which includes software e-commerce application. e.g. auto machines etc.

- Batch processing systems. large data process in batches e.g. salary system.

- Entertainment systems. personal use computers. e.g. computer games.

- Systems for modelling & simulation.

- Data system (collection) → Sensors

- systems of systems composed of no. of software.



Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

= Fundamental Principles:

- Managed & understood development process.
- Dependability & ^{performance} process → e.g. whatsapp..
- Software Specification And process Requirement
- Reuse Software.
- Cloud Computing (SaaS)

lec: 3

"SE"

4th October, 2024

'Software Processes:'

⇒ Steps:

1. Specification (Operation must be defined)
2. Design and Implementation (Specification must products)
3. Validation (checking that it's according to customer needs.)
4. Evolution (changing).

Process Descriptions:

⇒ It may include:

1. Products (outcomes of a process, like software architecture model)
2. Roles (Responsibilities assigned to peoples in the process)
3. Pre and Post conditions (After activity e.g. reviewed the model
Before an activity e.g. all requirements are approved).





Mo Tu We Th Fr Sa Su

Memo No. 1A

Date / /

Generic Process:

- progress tracked against the plan.
1. **Plan-driven Processes**. (All activities are planned upfront)
 2. **Agile Processes**. (Planning is flexible & adapts to changing) according to customer needs.
- ∴ No software process is universally wrong or right.

Generic Process Models: (No Risk)

1. **The waterfall model** (e.g of Plan-driven)
2. **Incremental development** (series of versions)
3. **Reuse-oriented Software** (it may be agile or plan-driven)

Issues Of Professional responsibility.

- | | |
|---|---|
| <ul style="list-style-type: none">• Confidentiality:
Engineers should respect clients & employer confidentiality, even without a formal agreement. | <ul style="list-style-type: none">• Intellectual property rights
Engineers must protect clients & employers intellectual properties. |
| <ul style="list-style-type: none">• Competence:
Should accurately represent their skills & avoid task beyond their expertise. | <ul style="list-style-type: none">• Computer Misuse.
Should avoid unauthorized or harmful computer use. |



Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

Coping with change

- change is inevitable in all large software projects.
- Change adds to the costs of software development.
- Change leads to rework both (rework and implementing new functionality)

* Reducing cost of Rework

- Change Avoidance e.g. prototype
- Change Tolerance (involve some incremental dev.)

* Two ways

1. System prototyping
2. Incremental delivery

* Software Prototyping

→ initial version of system.

→ Prototype can be used in

- requirement engineering process
- Design processes

* Benefits →

1. improve system usability
2. Achieve match of user needs

* Process

* Prototype development

3. improve design quality

* Throw-away prototypes

→ focus on functionality

4. improve maintainability
5. Reduced dev. effort



Lec: 4;

Software Process Activity ^{technical, collaborative & managerial}

→ Software Specification:

∴ elicitation → act of getting or producing something especially info/reaction

^{Report}
Feasibility study (technically & financially)

^{system model}
Requirement elicitation and analysis (expect from system)

^{stakeholders} ^{uses / system}
Requirement specification (further refine) user or system ^{req}

^{Documents}
Requirement validation (consistency & completeness) find errors

^{less details} ^{more details}
→ uses requirements / system requirements

→ Software design and implementation (development)

This activity are maybe interleaved

1. Software design

2. Implementation (Translate the structure into executable program)

— Design Input:

1. Platform Info (software & system or environment or challenge)

2. Requirement specification (Kya Karna Karega or Kitna fast or

3. Data Description (Data process / structure or details) ^{reliable} ^{hoga}

— Design Activity:

1. Architectural design

4. Database design

2. Interface design

3. Component design



Mo Tu We Th Fr Sa Su

Memo No.

Date

150

- Design Output:

1. System
2. Database

3. Interface
4. Component.

↳ Software Validation:

- Test Cases Use.
- Automated tools

- Test stages:

- Component testing (individual component Test)
- System testing (whole system Testing)
- alpha testing / Acceptance testing (with customer data)
- Beta testing (functionality temp. part)

↳ (To potential customers who agree to use that system).

Software Evolution:

Software is inherently flexible & can change.

Requirements change through business circumstances.

Testing Stages:

1. Component testing



2. System testing



3. Acceptance testing

lec : 5

'Incremental Delivery Advantages'

- ↳ Customer value → available earlier.
 - ↳ Elicit Requirements
 - ↳ highest priority system services
 - ↳ less Project failure
- Incremental development:
normally use agile methods.
develop develop & evaluate
by customer / user
increment delivery

Problems:

- ↳ It require a set of basic facilities
 - ↳ Requirement are not define in details
 - ↳ Conflicts with the procurement Model of many organization.
- Difficult to implement for
replaced system
→ risk-driven software process framework.

Boehm's Spiral Model: (Risk) (imp)

Spiral Model Sectors.

- ↳ Objective Setting (project risk identified) → Spiral model usage
- ↳ Risk assessment & Reduce. (reduced Key Risk) very influential in helping peoples.
- ↳ Development and validation.
- ↳ Planning (reviewed) can be any generic model maybe waterfall best RUP

The Rational Unified Process (UML and associate)

- ↳ Dynamic perspective (time) unified modeling
 - ↳ Static Perspective (activities) Language.
 - ↳ Practice perspective (process / more improvement)
- ↳ suggested good practices.

RUP Phases: (system design)

- ↳ **Inception**. (why we need the system / who will use it or interact with)
- ↳ **Elaboration**. (Understand problem / define req. - / Plan the project)
- ↳ **Construction**. (Build / test / document a fully functional ss)
- ↳ **Transitions**. (Deploy the system & ensure it works in its actual environment)
- ↳ **Documental Implementation**

RUP Iteration.

- ↳ **9th Phase** (Each phases progresses step by step, with results)
- ↳ **Cross phase** (build gradually)
- ↳ All Phases can be repeated incrementally as shown by the loop of RUP model.

Static View Of RUP:

1. focus on activity on development process.
2. workflows in RUP description.
3. Six core process workflows
4. 3 core process supporting workflows

Static Work flow (Table)

- | | |
|-----------------------|----------------------|
| 1. Business modeling. | 3. Analysis & design |
| 2. Requirements | 4. Implementation |
| 5. Testing | 6. Deployment |
| 7. Project management | 8. environment |



Mo Tu We Th Fr Sa Su

Memo No. _____

Date 12 / 10 / 24

lec: 6

Excellent ✨

Agile Software Development

jaldi deliver karna or new change to jaldi adapt karna iska

→ Agile Methode. (reduce the overheads) maqsad hai

- ^{divide} in small increments. (use iterative approach for fast delivery)
- focus on the code
- Rework though Plan-driven.

→ Agile Manifesto (strategy / Platform)

- Individuals and interactions. ^{imp then} (processes)
- Working software (more value than documentation)
- Customer collaboration (contract negotiation)
- Responding to change. (Plans flows karna)

→ Rapid Software development.

→ Principles of Agile Methods.

→ Agile Methode applicability.

- small or medium-sized projects (products)

→ Problem with Agile Method.

→ Agile Methods & software maintenance.

Key issues: 1. Documentation: (agr na ho to maintenance of ko mushkil bana sakta hai)

2. Customer change requests:

→ Technical, human, organizational issues.



Mo Tu We Th Fr ~~Sa~~ Su

Memo No.

Date 19 / 10 / 2024

lec: 7.

Extreme Programming →

is a popular agile method use in SD

- Requirements Scenarios. (user stories)
- A "prescribing medication" story (1. Fast Releases.
 - ↳ Define task / story card
 - 2. Testing
- XP & change (high) Refactoring. (code improvement)
- 3. Pair Programming
- 4. User Stories.

lec: 8

25/10/24

Waterfall | Incremental | Reuse | Bohem's | RUP

⇒ "Pair Programming"

- At one time two developers work on same project.
- less Risk.
- Efficient (high)

& Agile Project Management.

work on time



Mo Tu We Th **Fr** Sa Su

Memo No. _____

Date 25 / 10 / 25

(Product back log (list))

* **Scrum**: (Agile Project Ki Management Ka liya)

→ check progress.

→ 3 Phases:

↳ outline How, what and why, When etc.

main part ← **sprint cycles**. (planning unit) (sprint fix length 2-4 week)

↳ Project closure

→ Scrum Roles:

↳ Product owner.

↳ Scrum Master

↳ Development team. (3 to 9 members.)

* **Scaling Agile Methods:**

• **Large system Development:**

↳ Development time zada hota hai.

↳ how to communicate with shareholders.

• **Scaling UP & Scaling Out:**

(Agile) Large project.

large organization (expent)



Mo Tu We Th Fr S Su

Memo No. _____

Date 26 / 10 / 24

Requirement Engineering.

- requirement check
- Implementation check.
- Understand the problem.

∴ provide the services
that customer
requires
~~requires~~

∴ requirement engineering is the process of understanding customer needs. (problems it have).
& user

Main Topics:

Functional or non-functional requirements.

Requirement Specification

Requirement Documentation.

Requirement validation & so on (7 topic)

→ it may serve a dual function.
Requirement:

High-level abstract statement of a services of a system.
constraint to a detailed maths functional specification

Abstraction: Not in detail (just to the point) in start

After ~~start~~ decided explain in detail.

Types: ^{e.g. Q, problems} User Requirement: (simple services / constraints)

high-level abstraction ^{System Requirement}: (more detailed document)

e.g. input output test (Diff set of persons work on these)



Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

User Requirement: Client Manager, System end user, contract Manager
Client engineer, system Architect, software Developer.

System Requirement:

quality/standard & like.

apply on overall system / system services.

1. Functional & Non-Functional Requirement:

What system do and how to ; constraints apply on services

React on input / situation ; for standard quality

e.g: login

e.g: efficient, reliability.

Requirement Imprecision:

- When requirements are not precisely stated
- No ambiguity. Ambiguity: same requirement can mean different things
- Clearly Define. to users and developers.
- **NON Functional** (not directly concerned by specific service)
→ more critical than functional.

• User's Understanding:

• Developer's "

• Clear communication is essential to avoid such

Requirements Completeness & consistency.
misunderstanding

Complete: Requirement should cover all the functionality system need

Consistent: No conflicting & contradictory details

Reality: It is possible to create 100% complete / consistent requirement document.



Mo Tu We Th Fr Sa Su

Memo No. _____

Date 01 / 11 / 24

Lec: 10. "NON-FUNCTIONAL REQUIREMENTS."

how the system should work, not what it does.

[Functional] → Services | often more critical than
↳ Specific | functional requirements.

[Non-Functional] → constraints on services/properties
↳ General.

system properties: Reliability, response time, storage, processor speed etc

Constraints: Using specific programming language or IDE

[Classification:-

e.g. speed / Reliability

↳ Product Requirement (define how the product should behave

↳ Organizational "

↳ External " : from outside the system, like legal, regulatory or compliance rules (e.g: Central bank Rules)

↳ from company policies or prod procedures, such as using specific standard, programming language or how system operate.

Non-Functional Requirements.

Usability Requirements.

Domain Requirements (if it ^{aren't} met, system might not work properly.)

Train Protection System: → Formula.

$$(D_{\text{train}} = D_{\text{control}} + D_{\text{gradient}})$$

Domain requirement problem.

1. Understandability
2. Implicitness



Lec: 11

"The Software Requirement Document"

↳ what system we ~~make~~ build / what way we use.

~~Official document~~

= Official document of what (requirement) required of system development.

= It should include both def of ^{user} ~~system~~ requirement or Specification of system requirements

§ Agile Methods & Requirements:

- (use user stories) / this work well for business system
- (Skip detailed) / but harder for complex system or big teams

§ Users of a Requirement Document:

1. System Customer
2. Managers
3. System Engineers
4. System Test engineers
5. System Maintenance Engineers.

§ Requirement Document Variability

1. What type of system you build
2. What way you use to build

§ Requirement Specification:

• process of writing down the user & system requirement.

User: Write in a way that if end-user do not have a technical background he/she can understand easily

System: Give more details about the system.



Mo Tu We Th Fr Sa Su

Memo No. _____

Date 14 / /

'Structured Specification'

SS

→ form base. → ATM e.g. (Do not change easily)

→ Change in ~~natural~~ natural language

Ways of writing system requirement specification

1. Natural language.

written using numbered sentences

2. Structure Natural language:

written in natural language but on a template.

3. Graphical Notation:

Graphs / Models / diagrams.

4. Mathematical

equation / symbols.

* Guidelines for writing requirement

Problems with natural language.

- Lack of clarity
- Requirements confusion
- Requirement amalgamation.

& Form-Based Specification.

- | | |
|----------------|--------------------|
| 1. function | |
| 2. Description | 7. Action |
| 3. Inputs | 8. Requirement |
| 4. Source | 9. Pre-condition |
| 5. Output | 10. Post-condition |
| 6. Destination | 11. Side effects. |

& Tabular Specification

& Requirement Engineering Process.

- | | |
|----------------|---------------|
| 1. elicitation | 2. Analysis |
| 3. Validation | 4. Management |

& Requirement Discovery: (gathering info)

Documentation / System Stakeholders / Specification of similar system.

- | | |
|---|---|
| & Interviews in practices:
normally mix of
closed and open
interviewing. | & Interviewing:
• Closed interview:
• Open interview: |
|---|---|